

Fine-Tuning Flow: PEFT/LoRA & QLoRA for courtRoom.ai

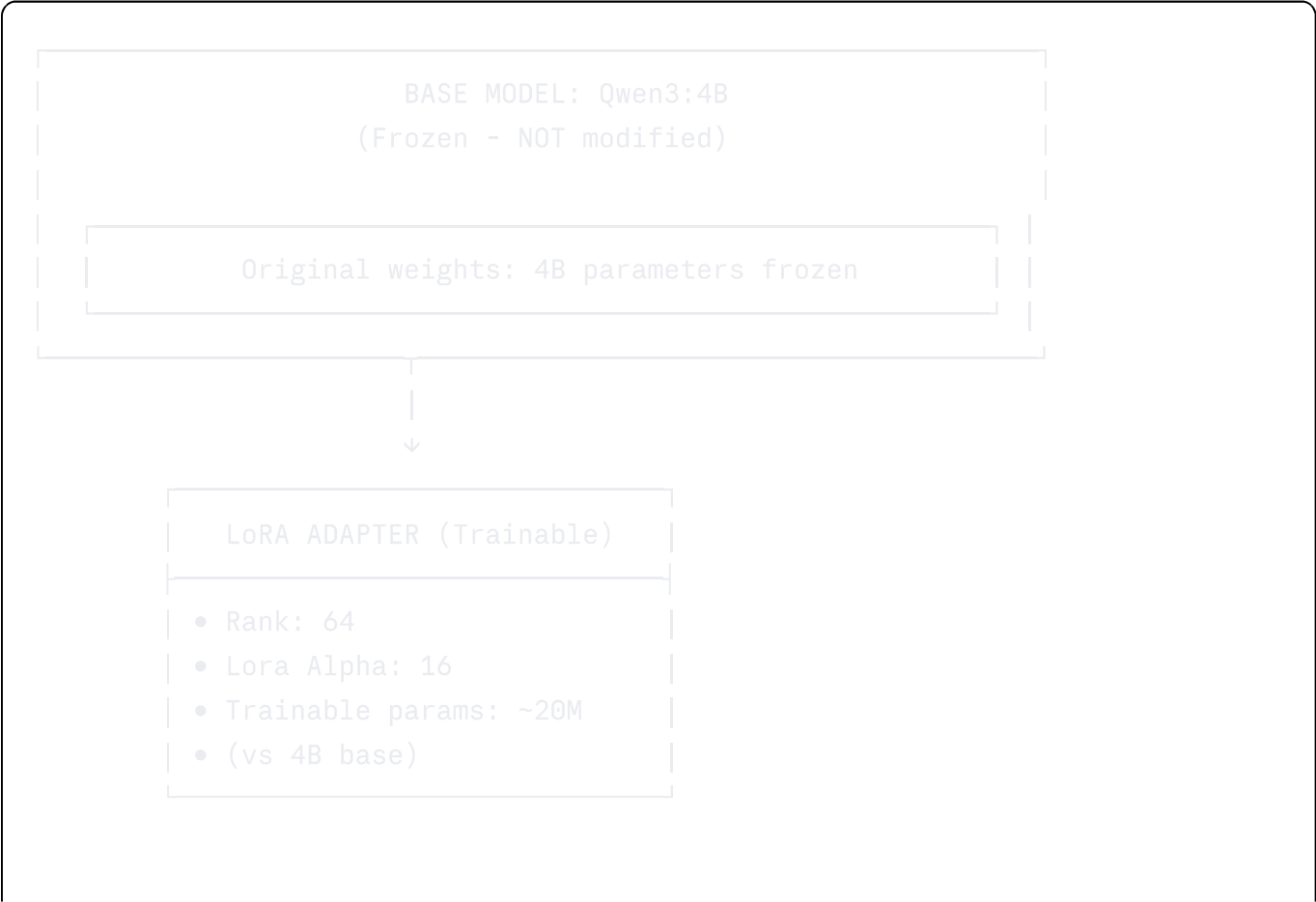
Overview

This guide walks you through fine-tuning **Qwen3:4B** using **PEFT (Parameter-Efficient Fine-Tuning)** with **LoRA** or **QLoRA** to create a legal domain-specific adapter for courtRoom.ai.

Why Fine-Tune?

- **Base Qwen3:4B** works well via RAG
- **Fine-tuned Qwen3:4B** with LoRA adapter specializes in Indian legal terminology, citation patterns, and case reasoning
- **QLoRA** saves GPU memory (4-bit quantization) — critical for Colab T4
- **Result:** Faster inference, more accurate legal citations, better chain-of-thought reasoning

Architecture: LoRA vs QLoRA



Example: query_proj, value_proj in attention layers
Original weight W ≈ 4B × 4 bytes = 16GB (FP32)
LoRA adapter ≈ 20M × 4 bytes = 80MB
Ratio: 0.5% trainable, 99.5% frozen

LoRA vs QLoRA Comparison

Aspect	LoRA	QLoRA
Base Model Precision	FP32 (full)	INT4 (4-bit) + NF4 quantization
Adapter Precision	FP32	FP32 (mixed)
GPU Memory	~8GB (T4)	~3-4GB (T4)
Training Speed	Fast	Slower (dequant overhead)
Accuracy	Baseline	Near-LoRA with less memory
Best For	Multiple adapters, fine-tuning	Memory-constrained (Colab)
Colab T4?	Tight but works	Recommended

Choice for courtRoom.ai: QLoRA — Colab T4 has 15GB total; 4B base + embeddings + data leaves ~4-6GB for training. QLoRA fits comfortably.

Step 1: Prepare Training Data

Data Format: Legal Q&A Pairs

```
python
# training_data.jsonl (one Q&A per line)
{
  "question": "My employer hasn't paid my salary for 2 months. What should I do",
  "answer": "Under the Payment of Wages Act, 1936 Section 15, your employer is
}
{
  "question": "I bought a defective product online. Can I get a refund?",
  "answer": "Under Consumer Protection Act 2019 Section 35, you have the right
}
...
```

Data Collection Process

1. Extract from courtRoom.ai logs (if running)
 - └ Successful inference engine rules → good Q&A pairs
 - └ User queries that reached RAG → questions
 - └ LLM responses → answers
2. Manual legal Q&A creation
 - └ Scrape public RTI responses
 - └ Consumer complaint forums (anonymized)
 - └ Labour dispute examples
 - └ Criminal case summaries
3. Synthetic generation (use base Qwen3 + RAG context)
 - └ Prompt: "Generate a realistic legal problem in [domain]"
 - └ Add retrieved law sections
 - └ Generate answer with section citations

Minimum Dataset Size

- **Domain-specific fine-tuning:** 300-500 examples (legal data is high-quality)
 - **Target split:** 80% train, 10% validation, 10% test
 - **Multi-domain:** Balance across Consumer, Labour, RTI, Criminal, Rent, Cyber
-

Step 2: Install PEFT + Dependencies

```
bash

# In Colab cell
!pip install peft>=0.7.0
!pip install bitsandbytes # Needed for QLoRA quantization
!pip install transformers>=4.36.0
!pip install torch torchvision torchaudio --index-url https://download.pytorch.
!pip install datasets
!pip install accelerate
!pip install trl # Trainer helpers

# Verify installations
python -c "import peft; print(peft.__version__)"
```

Step 3: Load Base Model in QLoRA Config

python

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

# ===== QUANTIZATION CONFIG =====
# Quantize base model to INT4 (NF4 format)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True, # Double quantization for extra memory sav
    bnb_4bit_quant_type="nf4",      # NF4 (Normalized Float 4) – superior to
    bnb_4bit_compute_dtype=torch.float16, # Computation dtype during training
)

# ===== LOAD BASE MODEL =====
model_name = "Qwen/Qwen3-4B" # Hugging Face model ID
# Note: If using ollama, export to HF format first:
# ollama pull qwen3:4b
# ollama export qwen3:4b path/to/qwen3.gguf
# Then convert GGUF → HF format using gguf-to-hf.py

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto", # Auto-place on available GPU
    trust_remote_code=True,
)

# ===== PREPARE FOR KBIT TRAINING =====
# Unfreezes certain params to allow training in quantized state
model = prepare_model_for_kbit_training(model)

# ===== LORA CONFIG =====
lora_config = LoraConfig(
    r=64, # LoRA rank (higher = more params, 64 is
    lora_alpha=16, # Scaling factor (alpha/r = 16/64 = 0.25
    target_modules=[
        "q_proj", # Query projection in attention
        "v_proj", # Value projection in attention
        "k_proj", # (Optional) Key projection
        "o_proj", # Output projection
    ],
    lora_dropout=0.05, # Dropout in LoRA layers for regularizat
    bias="none", # Don't add bias terms in LoRA

```

```
        task_type="CAUSAL_LM",                # Causal language modeling task
    )

# ===== APPLY LORA TO MODEL =====
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Output should show: trainable params ≈ 20M, total params ≈ 4B
```

Memory Check:

```
Before QLoRA:  4B model FP32 = ~16GB (won't fit T4)
After QLoRA:   4B model INT4 = ~1GB
LoRA adapter:  ~80MB
Embeddings:    ~2GB
Total used:    ~3-4GB (✓ fits T4 15GB with headroom)
```

Step 4: Prepare Data Loader

```
python
```

```

from transformers import AutoTokenizer
from datasets import load_dataset, Dataset
import json

# ===== TOKENIZER =====
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # Use EOS as padding

# ===== LOAD TRAINING DATA =====
# Option A: Load from JSONL file
def load_jsonl(file_path):
    data = []
    with open(file_path, 'r') as f:
        for line in f:
            data.append(json.loads(line))
    return data

train_data_raw = load_jsonl("training_data.jsonl")

# Option B: Convert to HF Dataset format
def format_qa_pair(example):
    """Convert Q&A pair to instruction-following format"""
    question = example["question"]
    answer = example["answer"]

    # Instruction template
    prompt = f"""Question: {question}

Answer: {answer}"""

    return {"text": prompt}

dataset = Dataset.from_dict({
    "text": [format_qa_pair(ex)["text"] for ex in train_data_raw]
})

# ===== TOKENIZE =====
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=512, # Legal texts are long, 512 tokens good balance
    )

```

```
        return_tensors=None,
    )

tokenized_dataset = dataset.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"]
)

# Split train/val/test
train_test = tokenized_dataset.train_test_split(test_size=0.2)
train_val = train_test["train"].train_test_split(test_size=0.1 / 0.8)

train_dataset = train_val["train"]
val_dataset = train_val["test"]
test_dataset = train_test["test"]

print(f"Train: {len(train_dataset)}, Val: {len(val_dataset)}, Test: {len(test_d")
```

Step 5: Configure Training Arguments

```
python
```



```

from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./legal_qwen_adapter",
    overwrite_output_dir=True,

    # ===== TRAINING SCHEDULE =====
    num_train_epochs=3,                # 3 passes through data
    per_device_train_batch_size=4,      # QLoRA can handle 4 on T4
    per_device_eval_batch_size=8,       # Eval batch larger (no backward)
    gradient_accumulation_steps=2,      # Accumulate 2 steps → eff. batch =

    # ===== OPTIMIZER =====
    learning_rate=2e-4,                # LoRA typically uses higher LR
    optim="paged_adamw_8bit",          # Memory-efficient AdamW for 8bit
    weight_decay=0.01,
    max_grad_norm=0.3,

    # ===== SCHEDULE =====
    warmup_steps=100,                  # Linear warmup
    lr_scheduler_type="cosine",        # Cosine annealing

    # ===== EVALUATION =====
    evaluation_strategy="steps",        # Eval every N steps
    eval_steps=50,                     # Check every 50 steps
    save_strategy="steps",
    save_steps=50,
    save_total_limit=3,                # Keep only 3 latest checkpoints
    load_best_model_at_end=True,

    # ===== LOGGING =====
    logging_steps=10,
    logging_dir="./logs",
    report_to="tensorboard",

    # ===== SYSTEM =====
    fp16=True,                         # Mixed precision training (FP16)
    gradient_checkpointing=True,        # Reduce memory during forward pass
    max_steps=500,                     # Stop after 500 steps (≈1-2 epochs)

    # ===== DISTRIBUTED =====

```

```
    ddp_find_unused_parameters=False,          # If using DDP
)
```

Step 6: Train with SFT Trainer

```
python

from trl import SFTTrainer # Supervised Fine-Tuning Trainer from trl library

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    peft_config=lora_config,
    dataset_text_field="text",
    max_seq_length=512,
)

# Start training
trainer.train()

# Evaluate on test set
eval_results = trainer.evaluate(eval_dataset=test_dataset)
print(f"Test Loss: {eval_results['eval_loss']:.4f}")
```

Typical Runtime:

- 300 examples $\times$ 3 epochs $\times$ 2 sec/batch $\approx$ 30 minutes on T4
 - 500 examples $\times$ 3 epochs $\approx$ 50 minutes
-

Step 7: Save & Merge Adapter

Option A: Save LoRA Adapter Only (Recommended)

```
python
```

```
# Save just the LoRA weights (~80MB)
model.save_pretrained("./legal_qwen_lora_adapter")
tokenizer.save_pretrained("./legal_qwen_lora_adapter")

# Later, load base model + adapter
from peft import AutoPeftModelForCausalLM

finetuned_model = AutoPeftModelForCausalLM.from_pretrained(
    "./legal_qwen_lora_adapter",
    device_map="auto",
)

# Merge into base (optional, makes inference faster)
merged_model = finetuned_model.merge_and_unload()
merged_model.save_pretrained("./legal_qwen_merged")
```

Option B: Merge Before Saving (Deployment)

```
python

# Merge LoRA weights into base model weights
merged_model = model.merge_and_unload()
merged_model.save_pretrained("./legal_qwen_merged_full")

# This creates a standalone 4B model, no need for adapter loading
# But file size: 4B params × 2 bytes (FP16) = 8GB (vs 80MB adapter)
```

Recommendation for courtRoom.ai:

- **Development:** Use adapter-only (lightweight, easy to iterate)
- **Production Colab:** Use adapter-only + load at startup
- **Export for offline:** Merge if storage not a constraint

Step 8: Inference with Fine-Tuned Model

```
python
```

```

from peft import AutoPeftModelForCausalLM
from transformers import AutoTokenizer

# Load fine-tuned model + adapter
model = AutoPeftModelForCausalLM.from_pretrained(
    "./legal_qwen_lora_adapter",
    device_map="auto",
    load_in_4bit=True, # Still quantized in inference
)
tokenizer = AutoTokenizer.from_pretrained("./legal_qwen_lora_adapter")

# Inference
question = "My employer hasn't paid salary for 2 months. What should I do?"
prompt = f"Question: {question}\n\nAnswer:"

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
outputs = model.generate(
    **inputs,
    max_new_tokens=512,
    temperature=0.1, # Low temperature for legal accuracy
    do_sample=False, # Greedy decoding (deterministic)
    top_p=0.95,
)

answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(answer)

```

Step 9: Integration with courtRoom.ai RAG

```
python
```

```
# In rag_pipeline.py / llm_router.py
```

```
class LLMRouter:
```

```
    def __init__(self, use_finetuned=True):
        self.use_finetuned = use_finetuned
```

```
    if use_finetuned:
```

```
        # Load fine-tuned adapter
```

```
        self.model = AutoPeftModelForCausalLM.from_pretrained(
            "./legal_qwen_lora_adapter",
            device_map="auto",
            load_in_4bit=True,
        )
```

```
        self.tokenizer = AutoTokenizer.from_pretrained(
            "./legal_qwen_lora_adapter"
        )
```

```
    else:
```

```
        # Fall back to base Qwen3:4B
```

```
        self.model = AutoModelForCausalLM.from_pretrained(
            "Qwen/Qwen3-4B",
            device_map="auto",
            load_in_4bit=True,
        )
```

```
        self.tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-4B")
```

```
    def generate_legal_response(self, query, retrieved_context):
```

```
        """
```

```
        Query: "My salary hasn't been paid for 2 months"
```

```
        Retrieved context: [Chunks from Payment of Wages Act Section 15, etc.]
```

```
        """
```

```
        prompt = self._build_prompt(query, retrieved_context)
```

```
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.model.device)
```

```
        outputs = self.model.generate(
```

```
            **inputs,
            max_new_tokens=512,
            temperature=0.1,
            do_sample=False,
        )
```

```
        response = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
        return response
```

```
def _build_prompt(self, query, context):
    return f"""Context (Relevant Law Sections):
{context}

User Query: {query}

Legal Response: """
```

Debugging QLoRA Issues

Issue 1: CUDA Out of Memory

```
python

# Solution 1: Reduce batch size
per_device_train_batch_size=2 # Down from 4
gradient_accumulation_steps=4 # Up from 2 (keep eff. batch ~8)

# Solution 2: Increase gradient checkpointing
gradient_checkpointing=True
```

Issue 2: Model Not Learning (Loss Plateaus)

```
python

# Check 1: Learning rate too high
learning_rate=1e-4 # Down from 2e-4

# Check 2: LoRA rank too low (underfitting)
r=128 # Up from 64

# Check 3: Verify data is in correct format
# Print first 3 examples:
for i in range(3):
    print(tokenizer.decode(train_dataset[i]["input_ids"][:100]))
```

Issue 3: Poor Inference Quality

```
python
```

```
# Check 1: Test on held-out examples
test_example = val_dataset[0]
print(tokenizer.decode(test_example["input_ids"]))

# Check 2: Verify fine-tuned model is loaded (not base)
print(model.peft_config) # Should show LoRA config

# Check 3: Adjust inference temperature
temperature=0.2 # Slightly higher for more creative answers
```

Optional: Advanced — Continue Training from Checkpoint

```
python

# If training interrupted, resume from last checkpoint
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
)

# Resume from checkpoint
trainer.train(resume_from_checkpoint="./legal_qwen_adapter/checkpoint-450")
```

Summary: LoRA Fine-Tuning Pipeline

- | |
|--|
| 1. PREPARE DATA |
| ├ Collect 300-500 legal Q&A pairs |
| ├ Format as JSONL |
| └ Split train/val/test |
| 2. SETUP QLoRA |
| ├ Load Qwen3:4B with INT4 quantization |
| ├ Create LoRA adapter (rank=64) |
| └ Prepare for kbit training |

3. TRAIN

- └ SFTTrainer: 3 epochs, batch=4, 50 min
- └ Monitor: eval loss, training loss

4. SAVE & DEPLOY

- └ Save adapter (~80MB)
- └ Load base + adapter at inference
- └ Integrate into RAG pipeline

Key Metrics to Track:

- Training loss (should decrease smoothly)
- Eval loss (should decrease, plateau $\approx 1.5-2.0$)
- Perplexity = $\exp(\text{eval_loss})$ (target < 20 for legal domain)

Next: Hybrid RAG Search Architecture

See `hybrid_rag_search_flow.md` for the complete vector + keyword search implementation.